

Estudo Comparativo de Estimadores de Estado num Pêndulo Invertido sob Malha de Controle

1 Introdução

Este projeto implementa e compara, **dentro de uma malha de controle fechada**, diferentes técnicas de **estimação de estados** aplicadas a um **pêndulo invertido sobre carro** (*cart-pole*) — um sistema dinâmico não-linear e naturalmente instável.

A motivação parte de uma lacuna do curso: em CMC-12 abordou-se apenas a **filtragem passa-baixas clássica**, sem estimação por espaço de estados. O trabalho avança sobre esse ponto confrontando quatro níveis de estimadores, do visto em aula ao estado da arte:

$$\underbrace{\text{Diferenciação + passa-baixas}}_{\text{baseline do curso}} \rightarrow \underbrace{\text{KF linear}}_{\text{linearizado}} \rightarrow \underbrace{\text{EKF}}_{\text{Jacobiano online}} \rightarrow \underbrace{\text{UKF}}_{\text{sigma points}} .$$

O pêndulo invertido é escolhido justamente porque o controle opera em **duas fases** que exercitam os filtros em regimes distintos: o *swing-up* (baseado em energia) leva a haste por **ângulos grandes**, num regime fortemente não-linear onde o KF linear degrada e EKF/UKF se destacam; já a *estabilização* (LQR na vertical) opera num regime quase-linear, onde todos os filtros convergem de forma semelhante. Assim, um único experimento evidencia **onde cada filtro compensa**.

Quanto ao escopo, o estado é $X = [x \ \dot{x} \ \theta \ \dot{\theta}]^T$ (posição e velocidade do carro; ângulo e velocidade angular da haste). Apenas as **posições** x e θ são medidas, com ruído (encoders); as velocidades $\dot{x}$ e $\dot{\theta}$ são estimadas. O desempenho é avaliado por RMSE por estado, tempo de convergência, robustez (ruído alto, baixa taxa de amostragem, erro de sintonia de Q/R , má inicialização), consistência estatística (NEES/NIS) e custo computacional, comparando ainda a malha fechada com estado estimado *vs.* estado ideal (princípio da separação na prática).

1.1 Parâmetros adotados

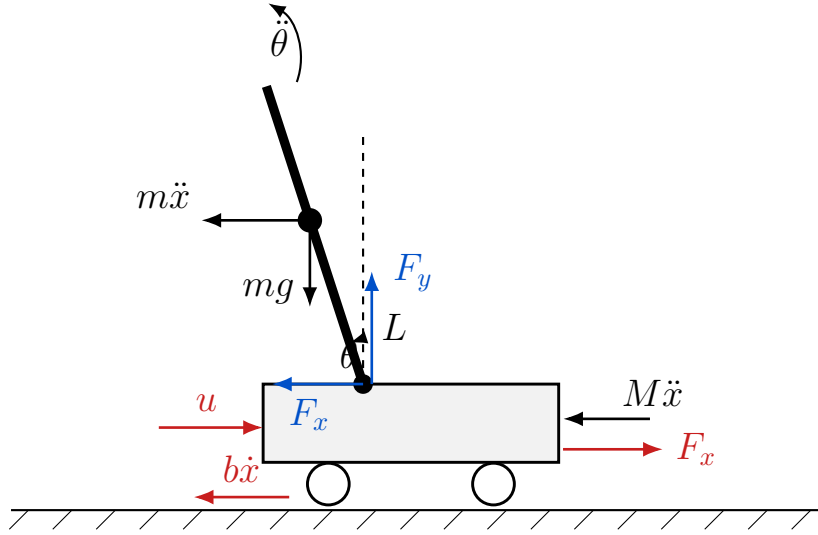
Todas as constantes físicas e de simulação são centralizadas em `params.m` (fonte única de verdade) e resumidas na Tabela 1. A haste é modelada como barra uniforme, de modo que o momento de inércia em torno do centro de massa é $I = \frac{1}{3}mL^2$.

Grandeza	Símbolo	Valor
Massa do carro	M	1,0 kg
Massa da haste	m	0,3 kg
Distância pivô-CM	L	0,5 m
Inércia da haste (CM)	$I = \frac{1}{3}mL^2$	0,025 kg·m ²
Atrito viscoso do carro	b	0,1 N·s/m
Gravidade	g	9,81 m/s ²
Passo de amostragem	Δt	0,01 s (100 Hz)
Horizonte de simulação	T_f	10 s (padrão)
Ruído do encoder (x)	σ_x	0,005 m
Ruído do encoder (θ)	σ_θ	0,005 rad

Tabela 1: Parâmetros físicos, de simulação e de medição adotados no projeto.

2 Planta de controle

No referencial não inercial do carrinho:



$$u + F_x - b\dot{x} = M\ddot{x} \quad (1)$$

$$mgL \sin \theta + m\ddot{x} L \cos \theta = (I + mL^2) \ddot{\theta} \quad (2)$$

$$F_x + m\ddot{x} = m \left(L\ddot{\theta} \cos \theta - L\dot{\theta}^2 \sin \theta \right) \quad (3)$$

Substituindo F_x da equação (3) na equação (1):

$$u + m \left(-\ddot{x} + L\ddot{\theta} \cos \theta - L\dot{\theta}^2 \sin \theta \right) - b\dot{x} = M\ddot{x}$$

Reorganizando, chega-se ao sistema:

$$\begin{cases} u = (M + m)\ddot{x} + b\dot{x} - mL\ddot{\theta}\cos\theta + mL\dot{\theta}^2\sin\theta \\ mL\ddot{x}\cos\theta = (I + mL^2)\ddot{\theta} - mgL\sin\theta \end{cases}$$

Escolhemos como vetor de estado

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix}, \quad \text{de modo que} \quad \dot{X} = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} x_2 \\ x_4 \\ x_3 \\ x_1 \end{bmatrix}.$$

Como $\dot{x}_1 = x_2$ e $\dot{x}_3 = x_4$ são imediatos, resta obter expressões explícitas para $\ddot{x}$ e $\ddot{\theta}$. Reescrevendo o sistema com os termos de aceleração à esquerda:

$$\begin{aligned} (M + m)\ddot{x} - mL\cos\theta\ddot{\theta} &= u - b\dot{x} - mL\dot{\theta}^2\sin\theta \\ -mL\cos\theta\ddot{x} + (I + mL^2)\ddot{\theta} &= mgL\sin\theta \end{aligned}$$

Podemos usar o formato matricial

$$\underbrace{\begin{bmatrix} M + m & -mL\cos\theta \\ -mL\cos\theta & I + mL^2 \end{bmatrix}}_A \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} u - b\dot{x} - mL\dot{\theta}^2\sin\theta \\ mgL\sin\theta \end{bmatrix}.$$

Para simplificar, definimos

$$\begin{aligned} a &= M + m, & h &= mL\cos\theta, & J &= I + mL^2, \\ r_1 &= u - b\dot{x} - mL\dot{\theta}^2\sin\theta, & r_2 &= mgL\sin\theta, \end{aligned}$$

de forma que $A = \begin{bmatrix} a & -h \\ -h & J \end{bmatrix}$.

O determinante é

$$\Delta = \det A = aJ - h^2 = (M + m)(I + mL^2) - m^2L^2\cos^2\theta,$$

e a inversa de uma matriz 2×2 simétrica é

$$A^{-1} = \frac{1}{\Delta} \begin{bmatrix} J & h \\ h & a \end{bmatrix}.$$

Logo,

$$\begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \frac{1}{\Delta} \begin{bmatrix} J & h \\ h & a \end{bmatrix} \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} \implies \begin{aligned} \ddot{x} &= \frac{Jr_1 + hr_2}{\Delta}, \\ \ddot{\theta} &= \frac{hr_1 + ar_2}{\Delta}. \end{aligned}$$

Substituindo a , h , J , r_1 e r_2 :

$$\ddot{x} = \frac{(I + mL^2)(u - b\dot{x} - mL\dot{\theta}^2 \sin \theta) + mL \cos \theta (mgL \sin \theta)}{(M + m)(I + mL^2) - m^2L^2 \cos^2 \theta},$$

$$\ddot{\theta} = \frac{mL \cos \theta (u - b\dot{x} - mL\dot{\theta}^2 \sin \theta) + (M + m)(mgL \sin \theta)}{(M + m)(I + mL^2) - m^2L^2 \cos^2 \theta}.$$

Para a haste uniforme, $I = \frac{mL^2}{3}$, de modo que $I + mL^2 = \frac{4}{3}mL^2$.

Reunindo tudo, com $X = [x \ \dot{x} \ \theta \ \dot{\theta}]^T$:

$$\dot{X} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} x_2 \\ \frac{(I + mL^2)(u - b x_2 - mL x_4^2 \sin x_3) + m^2 g L^2 \sin x_3 \cos x_3}{(M + m)(I + mL^2) - m^2 L^2 \cos^2 x_3} \\ x_4 \\ \frac{mL \cos x_3 (u - b x_2 - mL x_4^2 \sin x_3) + (M + m) mgL \sin x_3}{(M + m)(I + mL^2) - m^2 L^2 \cos^2 x_3} \end{bmatrix}.$$

3 Linearização em torno da vertical

O objetivo do controlador de estabilização é manter a haste na vertical ($\theta = 0$). Linearizamos, portanto, a dinâmica em torno do equilíbrio

$$X_{eq} = [0 \ 0 \ 0 \ 0]^T, \quad u_{eq} = 0,$$

usando as aproximações de pequenos ângulos e desprezando os termos de segunda ordem (o produto $\dot{\theta}^2 \sin \theta$ é de ordem superior e se anula):

$$\cos \theta \approx 1, \quad \sin \theta \approx \theta, \quad \dot{\theta}^2 \sin \theta \approx 0.$$

Nessas condições o determinante deixa de depender de θ (sua dependência é em $\cos^2 \theta$, cuja derivada em $\theta = 0$ é nula):

$$\Delta_0 = (M + m)(I + mL^2) - m^2L^2 = mL^2 \left[\frac{4}{3}(M + m) - m \right].$$

As expressões de $\ddot{x}$ e $\ddot{\theta}$ tornam-se lineares:

$$\ddot{x} \approx -\frac{(I + mL^2)b}{\Delta_0} \dot{x} + \frac{m^2L^2g}{\Delta_0} \theta + \frac{I + mL^2}{\Delta_0} u,$$

$$\ddot{\theta} \approx -\frac{mLb}{\Delta_0} \dot{x} + \frac{(M + m)mgL}{\Delta_0} \theta + \frac{mL}{\Delta_0} u.$$

Escrevendo $\dot{X} = AX + Bu$, obtemos a *matriz de estados* A e a *matriz de entrada* B :

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{(I + mL^2)b}{\Delta_0} & \frac{m^2L^2g}{\Delta_0} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{mLb}{\Delta_0} & \frac{(M + m)mgL}{\Delta_0} & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{I + mL^2}{\Delta_0} \\ 0 \\ \frac{mL}{\Delta_0} \end{bmatrix}.$$

O termo $A_{4,3} = (M + m)mgL/\Delta_0 > 0$ confirma que a vertical é um equilíbrio **instável**: um desvio $\theta > 0$ gera $\ddot{\theta} > 0$, afastando ainda mais a haste. É esse comportamento que a realimentação LQR corrige.

Instabilidade pelo critério de Routh-Hurwitz. Isolando o modo da haste (equação de $\ddot{\theta}$ linearizada), desprezando o acoplamento com o carro e o atrito e tomando $u = 0$, resulta $\ddot{\theta} - A_{4,3}\theta = 0$, cujo polinômio característico é

$$P(s) = s^2 + 0 \cdot s - A_{4,3}, \quad A_{4,3} = \frac{(M + m)mgL}{\Delta_0} = 17,795 > 0.$$

A *condição necessária* de estabilidade de Routh-Hurwitz exige que todos os coeficientes estejam presentes e tenham o mesmo sinal. Aqui ela já é violada em dois pontos: o coeficiente de s^1 está ausente ($a_1 = 0$) e o termo independente $a_2 = -A_{4,3}$ tem sinal oposto ao de $a_0 = 1$. Logo, $P(s)$ não pode ter todas as raízes no semiplano esquerdo.

Montando o arranjo de Routh (com a substituição $a_1 \rightarrow \varepsilon \rightarrow 0^+$ para o zero na primeira coluna):

$$\begin{array}{c|cc} s^2 & 1 & -A_{4,3} \\ s^1 & \varepsilon & 0 \\ s^0 & -A_{4,3} & \end{array}$$

A primeira coluna é $1, \varepsilon, -A_{4,3}$, isto é, $(+), (+), (-)$: ocorre **uma** troca de sinal, indicando **uma raiz no semiplano direito**. De fato, as raízes exatas são $s = \pm\sqrt{A_{4,3}} = \pm 4,22$, e a raiz $+4,22$ (modo $e^{+4,22t}$) é a que faz a haste cair. O termo $-A_{4,3}$ comporta-se como uma “constante de mola negativa”: é esse polo instável que a realimentação LQR reposiciona no semiplano esquerdo.

Valores numéricos. Com $M = 1,0$ kg, $m = 0,3$ kg, $L = 0,5$ m, $g = 9,81$ m/s², $b = 0,1$ N·s/m e $I = mL^2/3 = 0,025$ kg·m², tem-se $I + mL^2 = 0,1$ e $\Delta_0 = 0,1075$, resultando em

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -0,0930 & 2,0533 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -0,1395 & 17,795 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0,9302 \\ 0 \\ 1,3953 \end{bmatrix}.$$

4 Projeto do controlador LQR

A escolha do LQR se justifica pela própria natureza do problema. A planta é **instável** em malha aberta (Seção anterior, critério de Routh-Hurwitz): sem realimentação a haste inevitavelmente cai, de modo que um controlador ativo é obrigatório. Além disso, o cart-pole é um sistema de **quatro estados fortemente acoplados com uma única entrada** (u , a força no carro); técnicas clássicas de malha única (como um PID por variável) tornam-se difíceis de sintonizar nesse cenário multivariável, pois a mesma força precisa regular simultaneamente posição do carro e ângulo da haste. A realimentação de estados $u = -KX$ ataca todos os estados de uma vez, e o LQR fornece um critério **sistemático e ótimo** para escolher os quatro ganhos de K : em vez de posicionar polos por tentativa, minimiza-se um índice de desempenho que equilibra explicitamente o *erro de regulação* (via Q) contra o *esforço de controle* (via R). Como consequência, desde que o par (A, B) seja controlável, o LQR garante malha fechada estável e apresenta boas margens de robustez. Por fim, o LQR encaixa-se naturalmente no tema deste trabalho: pelo princípio da separação, o ganho projetado sobre o estado verdadeiro pode operar sobre o estado *estimado* ($u = -K\hat{X}$), permitindo comparar o desempenho da malha alimentada por cada estimador (KF, EKF, UKF).

4.1 Controlabilidade

Antes de projetar, verifica-se que o par (A, B) é controlável, isto é, que a matriz de controlabilidade tem posto completo:

$$\mathcal{C} = [B \ AB \ A^2B \ A^3B], \quad \text{posto}(\mathcal{C}) = 4.$$

Para o cart-pole isso se verifica: uma única força aplicada ao carro é capaz de governar os quatro estados.

4.2 Formulação do problema

O regulador linear quadrático escolhe a lei de realimentação de estados $u = -KX$ que minimiza o funcional de custo

$$J = \int_0^\infty (X^\top Q X + u^\top R u) dt, \quad Q = Q^\top \succeq 0, \quad R = R^\top \succ 0,$$

em que Q pondera o erro de estado e R pondera o esforço de controle. O ganho ótimo é

$$K = R^{-1}B^\top P,$$

onde $P = P^\top \succeq 0$ é a solução da equação algébrica de Riccati (ARE)

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

Para um sistema de quarta ordem a ARE não é resolvida à mão; obtém-se P (e daí K) numericamente (Seção 4.4).

4.3 Escolha de Q e R (regra de Bryson)

Como ponto de partida sistemático, aplica-se a *regra de Bryson*: cada peso é o inverso do quadrado do desvio máximo aceitável para aquela variável,

$$Q_{ii} = \frac{1}{x_{i,\max}^2}, \quad R = \frac{1}{u_{\max}^2}.$$

Adotando os limites físicos $x_{\max} = 0,5$ m, $\dot{x}_{\max} = 2$ m/s, $\theta_{\max} = 10^\circ \approx 0,175$ rad, $\dot{\theta}_{\max} = 2$ rad/s e $u_{\max} = 20$ N:

$$Q = \text{diag}(4; 0,25; 32,8; 0,25), \quad R = 0,0025.$$

Esses valores são apenas o ponto de partida; a sintonia final é iterativa (aumentar Q_{33} deixa o ângulo mais firme; aumentar R torna a resposta mais suave e econômica em força).

4.4 Cálculo do ganho no MATLAB

Com A , B , Q e R definidos, o ganho é obtido pela função `lqr` (requer a *Control System Toolbox*). O trecho abaixo reaproveita o Jacobiano analítico já implementado (`jacobian_f.m`) para montar A :

```
p = params(); % parametros fisicos
A = jacobian_f([0;0;0;0], 0, p); % matriz de estados (Jacobiano em theta=0)

d = p.m*p.L^2/3 + p.m*p.L^2; % I + mL^2
k = p.m*p.L; % mL
a = p.M + p.m; % M + m
Delta = a*d - k^2; % determinante em theta = 0
B = [0; d/Delta; 0; k/Delta]; % matriz de entrada

assert(rank(ctrb(A,B)) == 4, 'par (A,B) nao controlavel');

Q = diag([1/0.5^2, 1/2^2, 1/deg2rad(10)^2, 1/2^2]);
R = 1/20^2;

K = lqr(A, B, Q, R) % ganho otimo -> guardar em p.K_lqr
eig(A - B*K) % polos de malha fechada (Re < 0 => estavel)
```

O ganho resultante (valores de referência) é

$$K = [-40,00 \quad -42,14 \quad 217,20 \quad 48,46],$$

com polos de malha fechada

$$s_{1,2} = -12,8126 \pm 2,5904j,$$

$$s_{3,4} = -1,4435 \pm 1,0585j,$$

todos com parte real negativa, o que confirma a estabilização da vertical.

5 Controle de *swing-up* energético

O LQR só estabiliza a haste dentro de uma pequena vizinhança da vertical (sua região de atração). Quando o pêndulo parte pendurado, é preciso primeiro **levá-lo até essa vizinhança**. Como a linearização não vale para ângulos grandes, usa-se uma estratégia não-linear baseada em **energia** (*energy shaping*), seguindo a formulação de Tedrake [2] e o método clássico de Åström e Furuta [3]: em vez de seguir uma trajetória, injeta-se energia no pêndulo balançando o carro no ritmo certo, até que ele tenha energia suficiente para alcançar a vertical.

5.1 Energia do pêndulo

Adotamos a notação de [2]. A energia da haste é a soma das parcelas cinética T e potencial U ,

$$E(\theta, \dot{\theta}) = T(\dot{\theta}) + U(\theta), \quad T = \frac{1}{2}I \dot{\theta}^2, \quad U = -mgl \cos \theta,$$

onde I é o momento de inércia efetivo em torno do pivô e l a distância do pivô ao centro de massa. Para a haste uniforme deste trabalho, $I = \frac{1}{3}mL^2 + mL^2 = \frac{4}{3}mL^2$ e $l = L$.

Observação de convenção. A referência [2] mede θ a partir da posição **pendurada** ($\theta = 0$ embaixo), de modo que $U = -mgl \cos \theta$ é mínimo embaixo. Na modelagem deste relatório $\theta = 0$ é a **vertical superior**; as duas convenções diferem apenas por uma defasagem de π e um sinal no termo potencial, sem alterar a lei de controle.

A energia da órbita que alcança o equilíbrio superior instável (energia desejada) é, como em [2],

$$E^d = mgl.$$

O objetivo do *swing-up* é levar E até E^d . Define-se o erro de energia

$$\tilde{E} = E - E^d.$$

5.2 Lei de *energy shaping*

Para o pêndulo simples atuado por torque na junta, [2] mostra que a lei

$$u = -k \dot{\theta} \tilde{E}, \quad k > 0,$$

produz a dinâmica de erro $\dot{\tilde{E}} = -k \dot{\theta}^2 \tilde{E}$, que faz $\tilde{E} \rightarrow 0$. Isso decorre da função de Lyapunov $V = \frac{1}{2}\tilde{E}^2 \geq 0$, cuja derivada é

$$\dot{V} = \tilde{E} \dot{\tilde{E}} = -k \dot{\theta}^2 \tilde{E}^2 \leq 0.$$

No cart-pole a entrada não é um torque na junta, mas a **força** u no carro, que atua sobre a haste através da aceleração do pivô $\ddot{x}$. Da equação de movimento da haste, $I \ddot{\theta} = mgl \sin \theta + ml \cos \theta \ddot{x}$, obtém-se

$$\boxed{\dot{\tilde{E}} = ml \cos \theta \dot{\theta} \ddot{x}}$$

— os termos gravitacionais se cancelam e a energia só varia por meio de $\ddot{x}$. Repetindo o argumento de Lyapunov com $\dot{V} = \tilde{E} \dot{E} = \tilde{E} ml \cos \theta \dot{\theta} \ddot{x}$ e escolhendo $\ddot{x} \propto -\dot{\theta} \tilde{E} \cos \theta$, resulta $\dot{V} = -k ml \tilde{E}^2 \dot{\theta}^2 \cos^2 \theta \leq 0$. A lei de *energy shaping* adaptada ao cart-pole é, portanto, a de [2] com o fator geométrico $\cos \theta$:

$$u = -k \dot{\theta} \tilde{E} \cos \theta, \quad u \leftarrow \text{sat}_{\pm u_{max}}(u).$$

Quando falta energia ($\tilde{E} < 0$) a lei injeta energia ($\dot{E} > 0$); ao atingir E^d o termo se anula. Nos pontos de retorno ($\dot{\theta} = 0$) e na horizontal ($\cos \theta = 0$) tem-se $u = 0$.

5.3 Regulação da posição do carro

A lei de *energy shaping* pura tem uma limitação conhecida: ela governa a **energia da haste**, mas não faz nenhuma referência à **posição do carro**. Como o bombeamento empurra o carro sempre no sentido que adiciona energia, o carro deriva sem limite (nas simulações, dezenas de metros), e o LQR recebe a haste no topo com o carro deslocado — condição fora de sua região de atração, de modo que a captura falha.

A solução clássica, seguindo Chung e Spong [5] e Tedrake [4], é somar à lei de *energy shaping* um controlador **PD sobre o carro**, que o mantém próximo da origem sem impedir o balanço:

$$u = -k_{swing} \dot{\theta} \tilde{E} \cos \theta - \underbrace{k_p x - k_d \dot{x}}_{\text{PD no carro}}, \quad u \leftarrow \text{sat}_{\pm u_{max}}(u).$$

O termo $-k_p x$ age como uma mola que puxa o carro de volta a $x = 0$ e $-k_d \dot{x}$ como amortecimento. Optou-se por um PD (e não PI/PID): o *swing-up* é um transitório curto sob uma “perturbação” oscilatória (o próprio bombeamento troca de sinal a cada balanço), sem erro de regime permanente a corrigir — um termo integral apenas acumularia esse sinal (*windup*) e brigaria com o bombeamento. Os ganhos adotados, obtidos por busca em simulação, são

$$k_{swing} = 12, \quad k_p = 5, \quad k_d = 2,$$

com os quais a haste sobe e é capturada em torno de 1,2 s (estado ideal, sem ruído). Trata-se de um ponto de partida: os ganhos podem ser reotimizados (por exemplo, minimizando o tempo de subida sob restrição da excursão do carro).

5.4 Comutação para o LQR

O *swing-up* entrega a haste dentro da região de atração do LQR. Adota-se uma condição de comutação por ângulo e velocidade pequenos,

$$|\theta| < 15^\circ \quad \text{e} \quad |\dot{\theta}| < 2 \text{ rad/s} \implies \text{muda para } u = -K\hat{X},$$

combinando a fase não-linear (energia) com a fase linear (LQR) num único controlador de duas fases, exatamente o cenário que exercita os estimadores em regimes distintos.

6 Modelo de sensores e cenários de teste

O modelo de sensores é a interface entre a “verdade” do simulador e o que os estimadores efetivamente observam: recebe o estado verdadeiro e devolve uma medição imperfeita. Como os filtros nunca acessam X diretamente, é este modelo que torna o problema de estimação realista.

6.1 Grandezas medidas

Apenas as **posições** são medidas, reproduzindo um par de encoders na posição do carro e no ângulo da haste; as velocidades $\dot{x}$ e $\dot{\theta}$ não são medidas e devem ser estimadas. A medição é, portanto,

$$z_k = \begin{bmatrix} x \\ \theta \end{bmatrix}_{t_k} + v_k, \quad v_k \sim \mathcal{N}(0, R), \quad R = \text{diag}(\sigma_x^2, \sigma_\theta^2),$$

com ruído branco gaussiano e independente por canal. Os desvios nominais são $\sigma_x = \sigma_\theta = 0,005$ (Tabela 1). Essa escolha — medir só posições — é o que dá sentido à comparação de estimadores: obter $\dot{x}$ e $\dot{\theta}$ é exatamente a tarefa que separa a simples diferenciação com passagens baixas dos filtros baseados em modelo (KF/EKF/UKF).

6.2 Degradações e escolhas de projeto

Além do ruído nominal, o sensor reproduz três degradações, acionadas por parâmetros dos cenários (Seção de resultados). As escolhas de modelagem foram:

- **Ruído elevado (S3).** Multiplica-se σ_x e σ_θ por um fator ($10\times$). Testa a filtragem quando a relação sinal-ruído cai, favorecendo estimadores que ponderam corretamente medição *vs.* modelo.
- **Dropout de medida (S4).** A cada passo, com probabilidade p_{drop} , a leitura é **totalmente perdida**. Optou-se por sinalizar a perda com uma medição *vazia* ($z_k = \emptyset$), em vez de um valor qualquer, para que o estimador execute um passo de **predição pura** (sem atualização): a média é propagada pelo modelo e a covariância P cresce, refletindo corretamente o aumento de incerteza. Zerar ou congelar a medição daria ao filtro uma confiança falsa, corrompendo a análise de consistência (NEES/NIS).
- **Outliers.** Cada canal, com probabilidade p_{out} , recebe um pico espúrio de amplitude proporcional a σ (por padrão 30σ) e sinal aleatório. Modela leituras absurdas e ocasionais, que um estimador robusto deve rejeitar. É desligado por padrão ($p_{out} = 0$) e destina-se a um teste opcional de robustez.

Todas as degradações têm probabilidade nula por padrão, de forma que os cenários sem *override* usam apenas o ruído gaussiano nominal e permanecem comparáveis entre si.

7 Estimadores de Estado Clássicos: da Diferenciação ao Filtro de Kalman Linear

Esta seção descreve os três estimadores "clássicos" do projeto — isto é, os que não dependem de linearização *online* nem de amostragem estocástica do espaço de estados, ao contrário do EKF e do UKF, tratados na Seção 8. Os três compartilham a mesma interface de *drop-in* definida no contrato do projeto,

$$[\hat{X}_k, P_k] = \text{estimator_update}(\hat{X}_{k-1}, P_{k-1}, z_k, u_k, p),$$

e atuam sobre a mesma medição de posições $z_k = [x \ \theta]_{t_k}^T$ definida na Seção 6, mas diferem radicalmente na quantidade de **modelo** que incorporam: da ausência total de modelo dinâmico (baseline), passando por um modelo aproximado e implícito (complementar), até o modelo linear completo do pêndulo, explicitamente propagado com sua incerteza estatística (Kalman linear). Essa progressão prepara o terreno para o EKF e o UKF (Seção seguinte), que estendem o mesmo princípio de propagação de covariância ao modelo *não-linear* do cart-pole.

7.1 Baseline do curso: diferenciação numérica com filtro passa-baixas

O primeiro estimador (`est_lowpass.m`) reproduz exatamente a técnica vista em CMC-12: um filtro passa-baixas digital de primeira ordem (média móvel exponencial) aplicado à medição, seguido de diferenciação numérica para obter as velocidades [16]. Para cada canal medido,

$$\tilde{x}_k = \alpha \tilde{x}_{k-1} + (1 - \alpha) z_{x,k}, \quad \tilde{\theta}_k = \alpha \tilde{\theta}_{k-1} + (1 - \alpha) z_{\theta,k}, \quad \alpha \in (0, 1),$$

e as velocidades são obtidas por diferença atrasada (*backward difference*) do sinal já filtrado:

$$\hat{x}_k = \frac{\tilde{x}_k - \tilde{x}_{k-1}}{\Delta t}, \quad \hat{\theta}_k = \frac{\tilde{\theta}_k - \tilde{\theta}_{k-1}}{\Delta t}.$$

Esse filtro recursivo de primeiro grau é o análogo discreto de um passa-baixas *RC* contínuo com constante de tempo $\tau = -\Delta t / \ln \alpha$, ou, equivalentemente, frequência de corte $f_c = \frac{1}{2\pi\tau} = \frac{-\ln \alpha}{2\pi \Delta t}$ [16]. Com o valor padrão do código, $\alpha_{lp} = 0,8$, e $\Delta t = 0,01$ s (Tabela 1), obtém-se $f_c \approx 3,55$ Hz: qualquer conteúdo do sinal acima dessa frequência — inclusive parte da dinâmica real do *swing-up*, que é rápida e fortemente não-linear — é atenuado e atrasado junto com o ruído do encoder.

Esse é precisamente o ponto fraco do baseline, e a razão de ele servir de referência inferior na comparação: **não há nenhum modelo da planta** embutido no filtro. Quanto maior α , menor o ruído mas maior o atraso de fase (*lag*) — o mesmo compromisso ruído *vs.* atraso de qualquer filtro passa-baixas clássico [16] — e não existe forma de reduzir simultaneamente os dois, porque o filtro não distingue ruído de medição de dinâmica real do sistema. Note-se ainda que a covariância P não tem significado probabilístico aqui: o código apenas a repassa adiante ($P = P_{\text{prev}}$) só para respeitar a interface comum, já que não há modelo de incerteza a propagar.

7.2 Filtro complementar

O segundo estimador (`est_complementary.m`) já incorpora uma noção mínima de modelo: a cada passo, prediz a posição por integração de Euler à frente supondo velocidade constante,

$$x_{pred} = \hat{x}_{k-1} + \Delta t \hat{\dot{x}}_{k-1}, \quad \theta_{pred} = \hat{\theta}_{k-1} + \Delta t \hat{\dot{\theta}}_{k-1},$$

e funde essa predição com a medição bruta usando um ganho fixo $\alpha_{comp} \in (0, 1)$,

$$x_{fused} = (1 - \alpha_{comp}) x_{pred} + \alpha_{comp} z_{x,k}, \quad \theta_{fused} = (1 - \alpha_{comp}) \theta_{pred} + \alpha_{comp} z_{\theta,k},$$

com as velocidades novamente obtidas por diferença atrasada do sinal fundido. O nome "complementar" vem da interpretação clássica no domínio da frequência introduzida por Higgins [12]: dados dois sensores (ou, aqui, um sensor e um modelo) que medem a mesma grandeza com espectros de ruído complementares, pondera-se cada fonte por um par de filtros $H(s)$ e $1 - H(s)$ que somam a unidade —

$$\hat{X}(s) = H(s) Z(s) + (1 - H(s)) X_{pred}(s),$$

de modo que a medição (tipicamente confiável em baixa frequência, mas ruidosa em alta) e o modelo/predição (suave, mas sujeito a deriva em regime permanente) se complementam sem a necessidade de estimar covariâncias explicitamente. A versão discreta a ganho fixo implementada aqui é a forma mais simples dessa ideia; note-se que, com o ganho padrão do código, $\alpha_{comp} = 0,02$, o filtro confia fortemente na predição (98%) e corrige lentamente pela medição (2%) a cada passo — um compromisso adequado quando o modelo de velocidade constante é razoável entre amostras, mas que ainda assim acumula deriva se mantido por muitos passos sem medição (o mesmo problema clássico de INS/AHRS discutido por Higgins [12] e por Grewal *et al.* [9]). É justamente esse ganho fixo, não-ótimo e invariante no tempo, que o Filtro de Kalman generaliza na próxima subseção: o KF pode ser visto como um filtro complementar cujo ganho K_k é recalculado a cada passo para ser o *ótimo* dado o par modelo/medição [8, 13].

7.3 Do modelo linearizado ao Filtro de Kalman discreto

7.3.1 Discretização por segurador de ordem zero (ZOH)

O terceiro estimador (`est_kf.m`) usa o modelo *linear* $\dot{X} = AX + Bu$ já obtido na Seção 3 em torno da vertical, complementado pelo modelo de medição (linear e exato, já que só se observam posições diretamente),

$$z = CX, \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Como o KF discreto opera sobre o par (A_d, B_d) amostrado a Δt , e não sobre (A, B) contínuos, é preciso discretizar. A discretização exata de um sistema linear sob entrada constante por trecho (*zero-order hold*, ZOH) é

$$A_d = e^{A\Delta t}, \quad B_d = \int_0^{\Delta t} e^{A\tau} d\tau B,$$

resultado padrão de controle digital (ver, por exemplo, Franklin, Powell e Workman [14], ou Åström e Wittenmark [15]). Quando A é invertível, $B_d = A^{-1}(A_d - I)B$; **não é o caso aqui**: a dinâmica do cart-pole é invariante à translação (a posição x não aparece em nenhuma equação de movimento), de modo que a primeira coluna de A (Seção 3) é inteiramente nula e A é singular. O caminho numericamente robusto — e o que o MATLAB usa internamente em `c2d` — é formar a exponencial da matriz aumentada [14, 15]

$$\exp\left(\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} \Delta t\right) = \begin{bmatrix} A_d & B_d \\ 0 & I \end{bmatrix},$$

que evita a inversão e vale mesmo com A singular. Como a saída $z = CX$ é uma relação puramente algébrica (sem dinâmica), a discretização não altera C , ou seja, $C_d = C$. É exatamente esse procedimento que a função auxiliar `linearise_upright.m` executa via `c2d(ss(A_c,B_c,C_c,0),dt,'zoh')`, reaproveitando literalmente as matrizes A e B já derivadas e verificadas na Seção 3; o resultado $(p.Ad, p.Bd, p.Cd)$ é calculado uma única vez dentro de `params.m` e fica disponível a `est_kf.m` através do struct de parâmetros. A consistência entre esta linearização e o Jacobiano analítico de `jacobian_f.m` (Seção 8, derivado independentemente para o EKF) é verificada numericamente em `tests/test_classical_filters.m`, que bate as duas discretizações a menos de 10^{-8} (Seção 7.4).

Para conferência, com $\Delta t = 0,01$ s e os valores numéricos de A e B já apresentados na Seção 3, a discretização ZOH fornece (calculado numericamente a partir da matriz aumentada acima)

$$A_d \approx \begin{bmatrix} 1 & 0,009995 & 1,03 \times 10^{-4} & 3,4 \times 10^{-7} \\ 0 & 0,999070 & 0,020529 & 1,03 \times 10^{-4} \\ 0 & -6,98 \times 10^{-6} & 1,000890 & 0,010003 \\ 0 & -0,001395 & 0,177987 & 1,000890 \end{bmatrix}, \quad B_d \approx \begin{bmatrix} 4,65 \times 10^{-5} \\ 0,009298 \\ 6,98 \times 10^{-5} \\ 0,013951 \end{bmatrix}.$$

Como esperado de uma discretização fiel a um passo pequeno frente à dinâmica ($\Delta t = 0,01$ s $\ll$ constantes de tempo de A , cujos polos ficam em $\pm 4,22$ e $-0,077$ rad/s — Seção 3), A_d é próxima da identidade e os autovalores de A_d (isto é, $e^{\lambda_i(A)\Delta t}$) ficam próximos de 1: um deles, $e^{+4,2266 \times 0,01} \approx 1,0430$, maior que 1 em módulo, confirma que o modo instável da haste (Seção 3) permanece instável também no domínio discreto, como deveria.

7.3.2 Algoritmo do Filtro de Kalman

Com (A_d, B_d, C_d) em mãos, o KF linear implementado segue exatamente a forma padrão de predição-correção de Kalman [6, 7, 8]:

Predição (propaga estado e incerteza pelo modelo, sem usar a medição):

$$\hat{X}_k^- = A_d \hat{X}_{k-1} + B_d u_{k-1}, \quad P_k^- = A_d P_{k-1} A_d^\top + Q,$$

Atualização (corrige com a nova medição z_k):

$$S_k = C_d P_k^- C_d^\top + R, \quad K_k = P_k^- C_d^\top S_k^{-1}, \quad \hat{X}_k = \hat{X}_k^- + K_k (z_k - C_d \hat{X}_k^-), \quad P_k = (I - K_k C_d) P_k^-,$$

em que Q e R são as covariâncias (supostas conhecidas, diagonais e constantes) do ruído de processo e de medição, respectivamente (Tabela 2 adiante). O ganho K_k não é arbitrário: é

o único ganho que minimiza a incerteza remanescente $\text{tr}(P_k)$ a cada passo. Partindo da forma de Joseph $P_k = (I - K_k C_d) P_k^- (I - K_k C_d)^\top + K_k R K_k^\top$ (numericamente mais estável, adotada também no EKF do projeto), derivando $\text{tr}(P_k)$ em relação a K_k e igualando a zero obtém-se

$$\frac{\partial}{\partial K_k} \text{tr}(P_k) = 0 \implies K_k = P_k^- C_d^\top (C_d P_k^- C_d^\top + R)^{-1} = P_k^- C_d^\top S_k^{-1},$$

que é exatamente o ganho acima [6, 8]. Sob as hipóteses usuais — modelo linear, ruídos brancos, gaussianos, de média nula e descorrelacionados entre si e do estado inicial — esse ganho torna o KF o estimador de **variância mínima entre todos os estimadores lineares não enviesados** (propriedade BLUE, *Best Linear Unbiased Estimator*), e coincide com o estimador de máxima verossimilhança/MMSE quando os ruídos são gaussianos [6, 8, 9]. É essa otimalidade comprovada — e não apenas uma escolha heurística de ganho, como no filtro complementar (Seção 7.2) — que justifica o KF como o patamar de comparação "ótimo" *dentro do regime linear* para o restante do trabalho.

Escolha de Q e R . O código (`params.m`) adota, como ponto de partida,

Matriz	Valor
R (ruído de medição)	$\text{diag}(\sigma_x^2, \sigma_\theta^2) = \text{diag}(2,5 \times 10^{-5}, 2,5 \times 10^{-5})$
Q (ruído de processo)	$\text{diag}(10^{-4}, 10^{-3}, 10^{-4}, 10^{-3})$

Tabela 2: Covariâncias de ruído usadas pelo KF linear (compartilhadas com EKF/UKF via `p.Q/p.R`, salvo *override* `p.Q_kf/p.R_kf`).

R decorre diretamente do ruído físico dos encoders (Tabela 1) e não é, a rigor, um parâmetro de sintonia. Já Q modela a incerteza sobre o quanto o modelo *linearizado* deixa de representar a dinâmica real (erro de linearização, atrito não modelado, discretização, etc.) e é, na prática, um parâmetro de ajuste: Q pequeno demais faz o filtro confiar excessivamente no modelo linear e divergir fora da vizinhança da vertical (justamente o regime do *swing-up*); Q grande demais aproxima o KF de uma simples fusão ponderada, perdendo a vantagem de usar o modelo. Vale notar que o valor de Q aqui foi ajustado diretamente no tempo discreto, e não obtido pela discretização rigorosa de uma densidade espectral de ruído de processo contínuo (o que exigiria o método de Van Loan [17]); essa é uma simplificação comum em projetos aplicados [8], mas vale como ressalva ao interpretar Q como "física" em vez de "efetiva".

A consistência estatística desse ajuste de Q é o que o teste NEES (*Normalised Estimation Error Squared*), implementado em `compute_metrics.m`, mede: sob as hipóteses acima, $e_k^\top P_k^{-1} e_k$ segue uma distribuição χ^2 com 4 graus de liberdade (dimensão do estado); um NEES sistematicamente acima do intervalo esperado indica P **otimista** (subestima o erro real — tipicamente Q ou R pequenos demais), e um NEES sistematicamente abaixo indica P **pessimista** (superestima o erro — Q ou R grandes demais) [18]. O mesmo raciocínio, aplicado à inovação $\nu_k = z_k - C_d \hat{X}_k^-$ em vez do erro de estado, dá o teste NIS (*Normalised Innovation Squared*, χ^2 com 2 graus de liberdade — dimensão da medição), também implementado em `compute_metrics.m`.

Sensibilidade a Q/R mal ajustados (cenário S5). O cenário S5 testa exatamente esse ponto: o sensor continua gerando ruído nominal, mas os estimadores recebem covariâncias erradas, $Q_{\text{visto}} = 10Q$ e $R_{\text{visto}} = R/10$ (Seção 8 para a tabela completa com os cinco estimadores). O efeito é exatamente o previsto pela teoria: o KF confia mais na medição e menos na predição do que deveria, subestimando o erro real que sua própria covariância indica. O NEES médio do KF sobe de 1,66 em S1 (dentro do intervalo χ_4^2 a 95%, [0,48; 11,14]) para 19,19 em S5 — acima do limite superior, confirmando **P otimista**. Note-se que o RMSE de θ praticamente não piora ($0,24^\circ \rightarrow 0,29^\circ$): o ganho ainda funciona razoavelmente bem na prática, mas a filtro passa a **relatar** uma incerteza menor do que a real — o tipo de falha que só o teste de consistência estatística expõe, e que um RMSE sozinho esconderia.

7.4 Síntese comparativa

A Tabela 3 resume as diferenças estruturais entre os três estimadores desta seção.

Estimador	Modelo dinâmico	Ganho	P tem significado estatístico?
Baseline (passa-baixas)	Nenhum	Fixo (α_{lp})	Não
Complementar	Implícito (veloc. constante)	Fixo (α_{comp})	Não
KF linear	A_d, B_d (linearizado)	Ótimo, variável (K_k)	Sim (sob hipóteses lineares/gaussianas)

Tabela 3: Comparação estrutural dos três estimadores clássicos implementados.

Metodologia (resultado preliminar, pré-Fase 2). No momento em que este texto foi escrito, a planta não-linear (`plant_cartpole.m`), o controlador (`controller_lqr.m`, `swingup.m`) e o *dropout/outliers* do sensor (`sensor_model.m`) — todos de responsabilidade da Pessoa A — ainda eram *placeholders*, de modo que o harness completo (`main.m/run_scenario.m`) ainda não roda de ponta a ponta. Para validar o código desta seção e gerar resultados enquanto isso, foi escrito `tests/test_classical_filters.m`, que (i) confere numericamente a linearização de `linearise_upright.m` contra o Jacobiano analítico independente `jacobian_f.m` (Pessoa C) — as duas linearizações, feitas por pessoas diferentes, batem a menos de 10^{-8} ; (ii) simula uma trajetória próxima ao cenário S1 (haste a 5° da vertical) sob realimentação de estado *apenas para fins de demonstração*, usando o ganho K já publicado na Seção 4.4 — não é o `controller_lqr.m` definitivo, que continua em aberto; e (iii) roda os três estimadores desta seção em paralelo sobre essa trajetória, com o ruído nominal e com o multiplicador $10\times$ do cenário S3. Quando a Fase 2 do fluxo do projeto (README) estiver pronta, os mesmos gráficos devem ser regenerados a partir de `run_scenario.m` com os cenários reais S1/S3, o que pode alterar os números (mas não deve alterar as conclusões qualitativas abaixo, já que a única diferença relevante é o controlador de fato usado).

Resultados. A Tabela 4 e as Figuras 1–2 resumem o que foi observado.

Estimador	RMSE θ , S1 (°)	RMSE θ , S3 (°)	RMSE x , S1 (m)	RMSE x , S3 (m)
Baseline (passa-baixas)	0,250	1,018	0,0031	0,0175
Complementar	1,036	2,500	0,0093	0,0388
KF linear	0,241	1,157	0,0043	0,0188

Tabela 4: RMSE de θ e de x para os três estimadores clássicos, no cenário de demonstração descrito acima, com ruído nominal (S1) e com ruído $10\times$ (S3). Valores gerados por `tests/test_classical_filters.m`.

Dois pontos merecem destaque — um deles contraria a expectativa ingênua de que "mais modelo = sempre melhor":

- **Baseline e KF praticamente empatados, ambos à frente do complementar.** No regime quase-linear e bem amostrado (100 Hz) deste cenário, o passa-baixas bem ajustado ($\alpha_{lp} = 0,8$) e o KF linear entregam RMSE de θ quase idênticos ($0,25^\circ$ *vs.* $0,24^\circ$ com ruído nominal). Isso **não** contradiz a otimalidade do KF (Seção 7.3.2): o KF é ótimo *dado* o par (Q, R) configurado, e neste cenário de demonstração a trajetória verdadeira é gerada por uma EDO determinística, sem ruído de processo de fato injetado — ou seja, o Q de `params.m` (Tabela 2), calibrado para representar erro de linearização e dinâmica não modelada, é relativamente "generoso" frente ao ruído de processo real (próximo de zero) deste cenário específico. Isso dá ao ganho do KF uma folga maior do que o estritamente necessário, deixando passar um pouco mais de ruído de medição do que um passa-baixas fortemente suavizante deixaria — o mesmo fenômeno de sensibilidade a Q/R misturado que o cenário S5 é desenhado para expor (Seção 7.3.2). A vantagem estrutural do KF (explorar A_d, B_d) deve ficar mais evidente quando houver ruído de processo real (p.ex. erro de linearização acumulado fora da vizinhança da vertical) ou quando Q for recalibrado — ambos fora do escopo desta seção.
- **Filtro complementar é o pior dos três, por larga margem.** A Figura 1 mostra o motivo: com o ganho padrão $\alpha_{comp} = 0,02$ (Seção 7.2), o filtro confia 98% na predição de velocidade constante a cada passo e corrige lentamente pela medição — adequado para deriva lenta, mas lento demais para acompanhar a oscilação inicial rápida de 5° deste cenário: o filtro *overshoots* a mais de 6° e só converge por volta de $t \approx 2$ s, enquanto baseline e KF acompanham a verdade quase imediatamente. Sob ruído $10\times$ (S3), seu RMSE mais que dobra ($1,04^\circ \rightarrow 2,50^\circ$), a maior degradação relativa das três — esperado, já que α_{comp} é fixo e não se adapta ao novo nível de ruído, ao contrário do ganho K_k do KF, recalculado a cada passo.

Nenhum dos três estimadores desta seção foi testado acima no *swing-up* (cenário S2, ângulos grandes): no momento em que este experimento de demonstração foi montado, o controlador de *swing-up* (Pessoa A) ainda era um *placeholder*, e a realimentação usada acima é sobre o **estado verdadeiro** ($u = -K_{demo}X$), não sobre a estimativa — um recurso deliberado para poder testar os três filtros antes de o restante da malha estar pronto (`tests/test_classical_filters.m`).

É precisamente no swing-up que a linearização por trás do KF deixa de valer; a comparação dos cinco estimadores nesse regime, com o harness completo e a malha fechada real ($u = -K\hat{X}$ via `swingup.m`), está na Seção 8.

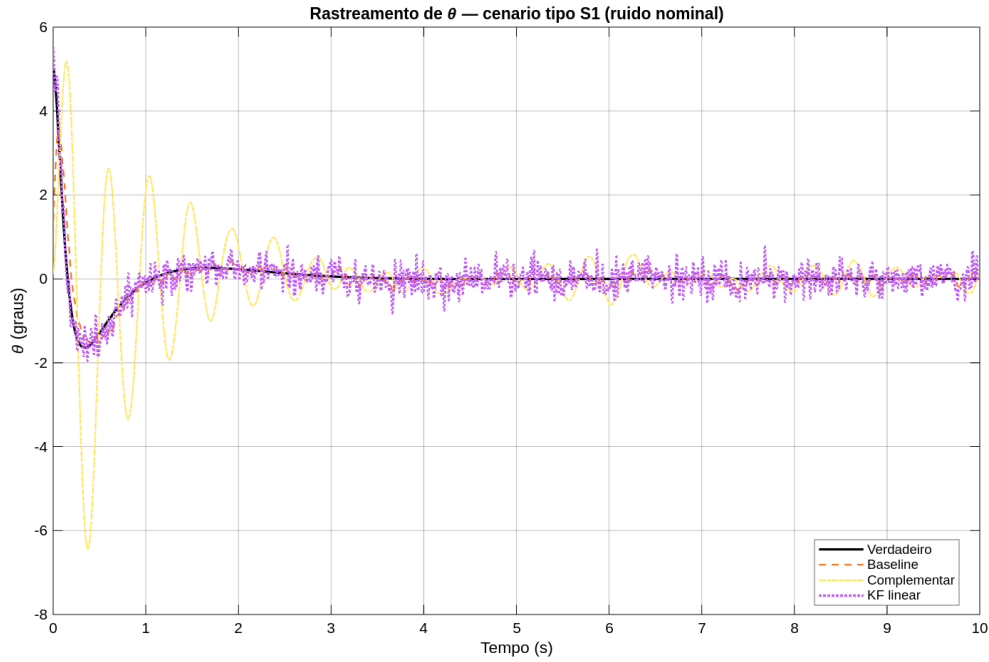


Figura 1: Ângulo θ estimado pelos três estimadores clássicos no cenário de demonstração (análogo a S1, 5° de perturbação inicial, ruído nominal) *vs.* estado verdadeiro simulado. O filtro complementar (ganho fixo $\alpha_{comp} = 0,02$) *overshoota* e leva ~ 2 s para convergir; baseline e KF acompanham a verdade quase imediatamente.

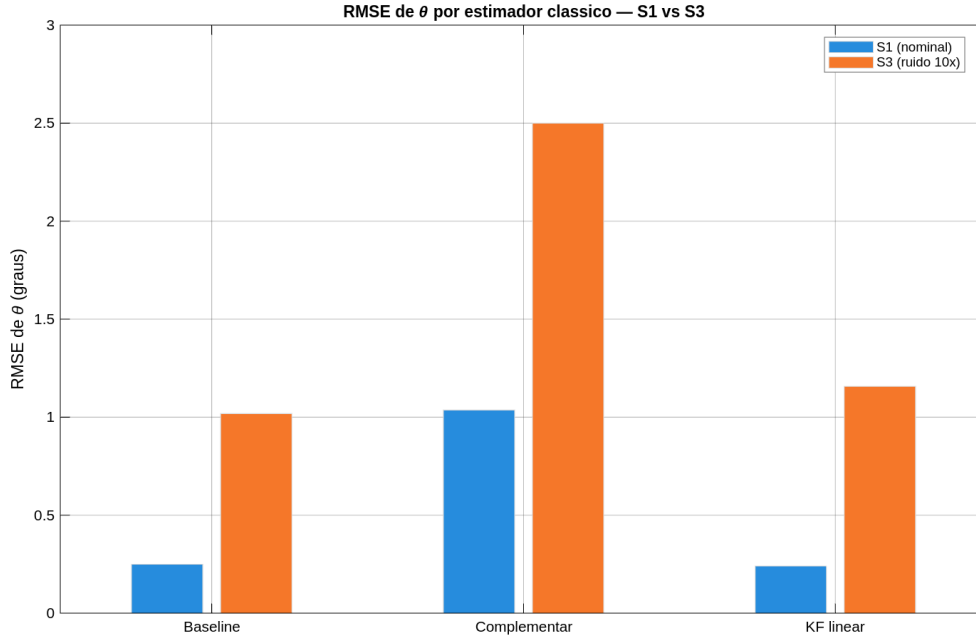


Figura 2: RMSE de θ dos três estimadores clássicos no cenário de demonstração, com ruído nominal e com ruído $10\times$ (análogo a S1 vs. S3). O complementar tem a maior degradação relativa; baseline e KF permanecem próximos em ambos os níveis.

Da demonstração à Fase 2. A planta não-linear (`plant_cartpole.m`), o sensor com *dropout/outliers* (`sensor_model.m`), o LQR (`controller_lqr.m`) e o *swing-up* (`swingup.m`) — todos de responsabilidade da Pessoa A — estão hoje implementados e integrados por `run_scenario.m/main.m` (`src/main.m` roda os cinco estimadores nos seis cenários S1–S6 e salva `results/*.mat`), de modo que a Fase 2 do fluxo do projeto (README) está completa. A Seção 8 apresenta os resultados dessa malha real — inclusive uma diferença importante em relação à demonstração acima: ali o ganho K_{demo} realimentava o **estado verdadeiro**; na malha real $u = -K\hat{X}$ realimenta a **estimativa**, de modo que um estimador ruim degrada o próprio controle (e não só o RMSE do estimador isolado) — o princípio da separação em ação, e não apenas em teoria.

8 Estimadores de Estado Não-Lineares: EKF e UKF

O KF linear (Seção 7.3) é ótimo *dado* o modelo (A_d, B_d, C_d) , mas esse modelo é uma linearização fixa em torno de um único ponto de operação, $\theta = 0$. Fora da vizinhança da vertical — precisamente o regime do *swing-up* (Seção 6, cenário S2) — a dinâmica verdadeira do cart-pole diverge da dinâmica linear, e nada garante que o KF continue sequer estável, muito menos ótimo. Esta seção trata os dois estimadores que lidam com a não-linearidade explicitamente: o **Filtro de Kalman Estendido** (EKF), que relineraliza a cada passo em torno da própria estimativa corrente, e o **Filtro de Kalman Unscented** (UKF), que dispensa qualquer linearização analítica e propaga a incerteza por meio de um conjunto determinístico de pontos (*sigma points*) através do modelo não-linear exato. Os dois seguem a mesma interface *drop-in* das seções anteriores,

$[\hat{X}_k, P_k] = \text{estimator_update}(\hat{X}_{k-1}, P_{k-1}, z_k, u_k, p)$, e ambos expõem opcionalmente um terceiro argumento de saída $\text{dbg} = (\nu_k, S_k)$ com a inovação e sua covariância, usado pelo teste de consistência NIS em `compute_metrics.m`.

8.1 Filtro de Kalman Estendido (EKF)

8.1.1 Jacobiano analítico

O EKF precisa, a cada passo, de uma aproximação linear local da dinâmica em torno da estimativa corrente $\hat{X}_{k-1}$. Em vez de linearizar apenas uma vez (como em 3), `jacobian_f.m` calcula $F = \partial f / \partial X$ **analiticamente, em qualquer estado**, reaproveitando a mesma solução do sistema 2×2 de `plant_cartpole.m` (Seção 3, $\Delta = aJ - h^2$) e aplicando a regra do quociente às expressões de $\ddot{x}$ e $\ddot{\theta}$. Como a dinâmica não depende de x (invariância translacional, já observada na Seção 7.3.1), a primeira coluna de F é sempre $[0, 1, 0, 0]^\top$ /nula conforme a linha; o restante $(F_{2,2}, F_{2,3}, F_{2,4}, F_{4,2}, F_{4,3}, F_{4,4})$ é avaliado em $(\hat{x}_2, \hat{x}_3, \hat{x}_4, u)$ a cada chamada. A corretude desse Jacobiano é verificada numericamente (Seção 8.3) contra diferenças finitas centrais da própria `plant_cartpole.m` — e, de forma independente, contra a linearização de `linearise_upright.m` no ponto $\theta = 0$ (Seção 7.4), já que as duas foram derivadas separadamente pelas Pessoas B e C a partir das mesmas equações de movimento.

8.1.2 Predição: integração não-linear + covariância linearizada localmente

Diferentemente do KF, cuja predição é uma multiplicação matricial fixa $(A_d \hat{X}_{k-1} + B_d u_{k-1})$, o EKF integra a **dinâmica não-linear completa** (RK4, mesmo integrador da planta verdadeira) para propagar a média:

$$\hat{X}_k^- = \text{RK4}(f, \hat{X}_{k-1}, u_{k-1}, \Delta t),$$

e usa o Jacobiano *congelado* em $\hat{X}_{k-1}$, discretizado pela mesma exponencial de matriz da Seção 7.3.1 (aqui sem termo de entrada, já que só a covariância é propagada dessa forma),

$$F = \text{jacobian_f}(\hat{X}_{k-1}, u_{k-1}, p), \quad F_d = e^{F \Delta t}, \quad P_k^- = F_d P_{k-1} F_d^\top + Q.$$

A média, portanto, segue a física exata (sem erro de linearização acumulado); só a incerteza é propagada por uma aproximação linear local — e essa aproximação é **recalculada a cada passo** no ponto atual, ao contrário do (A_d, B_d) fixo do KF. É essa relinearização contínua que permite ao EKF acompanhar o *swing-up* sem perder a estimativa, como mostrado na Seção 8.4.

8.1.3 Atualização

O modelo de medição continua exato e linear ($z = CX$, mesma matriz C da Seção 7.3, já que só posições são observadas), de modo que a atualização tem a mesma forma do KF, na forma de Joseph (numericamente estável, adotada também pelo KF linear — Seção 7.3.2):

$$S_k = C P_k^- C^\top + R, \quad K_k = P_k^- C^\top S_k^{-1}, \quad \hat{X}_k = \hat{X}_k^- + K_k \nu_k, \quad \nu_k = z_k - C \hat{X}_k^-,$$

$$P_k = (I - K_k C) P_k^- (I - K_k C)^T + K_k R K_k^T.$$

Quando a medição é perdida (*dropout*, S4 — sinalizada por $z_k = [\text{NaN}, \text{NaN}]^T$), o EKF pula a atualização e retorna $(\hat{X}_k^-, P_k^-)$: a mesma convenção de predição-pura adotada pelo KF linear e pelos estimadores clássicos (Seção 6).

8.2 Filtro de Kalman Unscented (UKF)

8.2.1 Motivação: aproximar a distribuição, não a função

O EKF resolve a não-linearidade linearizando a *função* de transição a cada passo — uma aproximação de primeira ordem (série de Taylor truncada) que descarta termos de ordem ≥ 2 . Quando a dinâmica é fortemente não-linear num único passo de integração (curvatura grande de f na vizinhança de $\hat{X}_{k-1}$), esse truncamento introduz um viés que se acumula. A ideia do UKF [10, 11], ao contrário, é que é mais fácil aproximar uma **distribuição de probabilidade** do que uma **função** não-linear arbitrária: em vez de linearizar f , escolhe-se deterministicamente um pequeno conjunto de pontos (*sigma points*) que reproduzem exatamente a média e a covariância de $\hat{X}_{k-1}$, propaga-se **cada um deles pela dinâmica não-linear exata** (a mesma `plant_cartpole.m`/RK4 usada pela planta verdadeira, sem aproximação alguma), e reconstrói-se a média e a covariância a posteriori a partir dos pontos já transformados. O resultado é exato até 2^a ordem da série de Taylor da média (3^a ordem se a distribuição de entrada for gaussiana), uma ordem a mais que o EKF, e sem exigir nenhum Jacobiano.

8.2.2 Sigma points e pesos (transformada *unscented* escalonada)

A implementação (`est_ukf.m`) segue a forma escalonada de Van der Merwe [11]. Para um estado de dimensão $n = 4$, três parâmetros de ajuste controlam a dispersão dos pontos: α (tipicamente pequeno, 10^{-3} , controla o espalhamento em torno da média), β (incorpora conhecimento prévio sobre a distribuição; $\beta = 2$ é ótimo para gaussianas) e κ (parâmetro secundário de escala, geralmente 0). Define-se

$$\lambda = \alpha^2(n + \kappa) - n,$$

e os $2n + 1 = 9$ *sigma points* são a média mais/menos as colunas da raiz de Cholesky da covariância escalada,

$$\mathcal{X}_0 = \hat{X}_{k-1}, \quad \mathcal{X}_i = \hat{X}_{k-1} \pm \left(\sqrt{(n + \lambda) P_{k-1}} \right)_i, \quad i = 1, \dots, n,$$

com pesos de média e de covariância

$$W_0^{(m)} = \frac{\lambda}{n + \lambda}, \quad W_0^{(c)} = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta), \quad W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n + \lambda)} \quad (i \geq 1).$$

Se a fatoração de Cholesky de P_{k-1} falhar (não positiva-definida, o que pode ocorrer numericamente sob covariâncias iniciais extremas como no cenário S6), o código regulariza somando $10^{-6}I$ antes de tentar novamente (`est_ukf.m`).

8.2.3 Predição

Cada um dos $2n + 1$ *sigma points* é propagado pela dinâmica não-linear completa (RK4, sem aproximação), e a média e a covariância previstas são reconstruídas por combinação ponderada:

$$\mathcal{X}_i^- = \text{RK4}(f, \mathcal{X}_i, u_{k-1}, \Delta t), \quad \hat{X}_k^- = \sum_{i=0}^{2n} W_i^{(m)} \mathcal{X}_i^-,$$

$$P_k^- = Q + \sum_{i=0}^{2n} W_i^{(c)} (\mathcal{X}_i^- - \hat{X}_k^-) (\mathcal{X}_i^- - \hat{X}_k^-)^\top.$$

8.2.4 Atualização

Em geral, o UKF propagaria os *sigma points* também através do modelo de **medição** $h(\cdot)$ para lidar com sensores não-lineares. Aqui, porém, $h(X) = CX$ é **linear** (só posições são observadas — mesma matriz C do KF/EKF), de modo que propagar os pontos por h e recombinar produziria *exatamente* $CP_k^-C^\top + R$ — o mesmo resultado que a forma fechada do KF, sem nenhuma perda de precisão. A implementação explora essa igualdade algébrica diretamente (mais barato, sem reamostrar os pontos):

$$S_k = CP_k^-C^\top + R, \quad K_k = P_k^-C^\top S_k^{-1}, \quad \hat{X}_k = \hat{X}_k^- + K_k \nu_k, \quad \nu_k = z_k - C\hat{X}_k^-,$$

$$P_k = (I - K_k C) P_k^-.$$

Diferentemente do KF e do EKF, esta última atualização *não* usa a forma de Joseph — como a álgebra acima é exata (não há linearização nesta etapa, h já é linear), o ganho de estabilidade numérica da forma de Joseph é menor aqui; ainda assim, adotá-la uniformemente nos três filtros seria uma simplificação de manutenção razoável para trabalhos futuros. O caso de medição perdida (*dropout*) segue a mesma convenção dos demais estimadores: sem atualização, retorna-se $(\hat{X}_k^-, P_k^-)$.

8.3 Verificação numérica

`tests/test_nonlinear_filters.m` valida três propriedades antes de qualquer resultado de desempenho ser levado a sério:

1. **Jacobiano analítico vs. diferença finita central.** Para seis valores de θ (incluindo $0, \pm\pi/4, \pm\pi/2$ e π), o erro máximo entre `jacobian_f.m` e uma diferença finita central de `plant_cartpole.m` (passo $h = 10^{-6}$) foi $2,51 \times 10^{-9}$, muito abaixo da tolerância de 10^{-6} — confirma que a álgebra do Jacobiano (regra do quociente aplicada a $\ddot{x}, \ddot{\theta}$) está correta em todo o domínio de θ , não só perto da vertical.
2. **Smoke tests de numérica.** Ambos os filtros, em duas trajetórias sintéticas (oscilação amortecida perto de $\theta = \pi$ e um balanço violento de laço aberto), mantiveram $\hat{X}$ e P

finitos, P simétrica e positiva-semidefinida (via `chol`) em todos os passos, e o erro nos estados observados (x, θ) permaneceu perto do piso de ruído do encoder mesmo atravessando medições perdidas (*dropout*) sem produzir NaN/Inf.

3. **Interface.** As duas chamadas — com e sem o terceiro argumento de depuração `dbg` — funcionam; `dbg` tem o formato esperado ($\nu \in \mathbb{R}^2$, $S \in \mathbb{R}^{2 \times 2}$) e vem vazio quando a medição é perdida (sem atualização, sem inovação a reportar).

Todos os sete testes passam (ALL TESTS PASSED).

8.4 Resultados: divergência do KF fora do regime linear (*swing-up*, S2)

Com a Fase 2 completa (planta, sensor, controlador e harness da Pessoa A — Seção 7.4), é possível finalmente rodar os cinco estimadores no cenário que mais importa para este trabalho: o *swing-up* (S2), em que a haste passa por $\theta \in (-\pi, \pi]$ antes de ser capturada pelo LQR — exatamente o regime para o qual a linearização do KF em $\theta = 0$ não foi projetada.

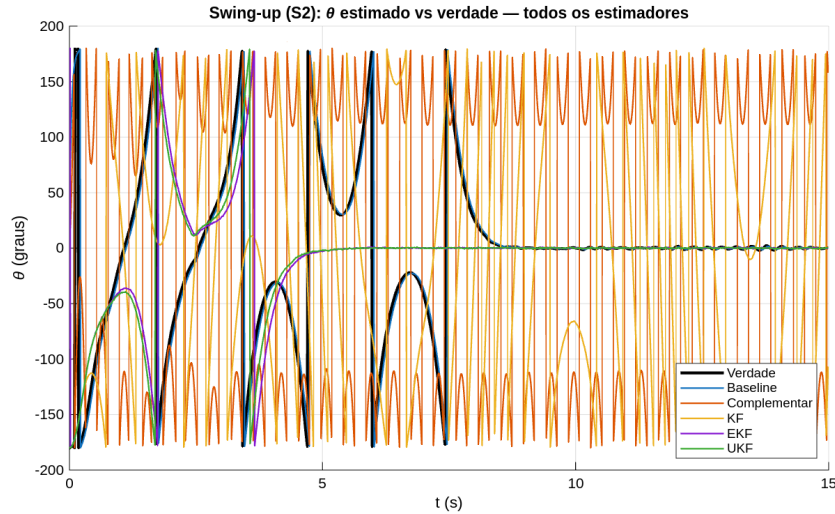


Figura 3: Ângulo θ verdadeiro *vs.* estimado pelos cinco estimadores durante o *swing-up* completo (S2, malha fechada real com `swingup.m`). Baseline e KF acompanham a verdade razoavelmente bem enquanto o pêndulo ainda executa poucos balanços, mas o KF passa a oscilar sem relação com a verdade assim que o ângulo cruza regiões distantes de $\theta = 0$; EKF e UKF (roxo/verde) colam na verdade (preto) do início ao fim, inclusive durante os balanços de 180° .

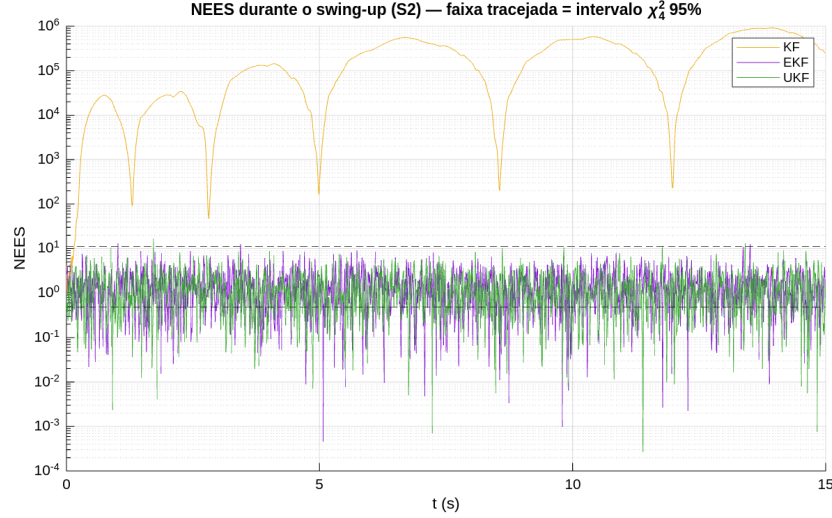


Figura 4: NEES ao longo do *swing-up* (S2) para KF, EKF e UKF (escala log). A faixa tracejada é o intervalo χ_4^2 a 95%, $[0,48; 11,14]$. O KF passa a maior parte do tempo **quatro a cinco ordens de grandeza** acima do limite superior — o próprio filtro "acredita" ter convergido (sua covariância P encolhe) ao mesmo tempo em que o erro real explode, o padrão clássico de **divergência otimista** de um filtro linear fora de sua região de validade. EKF e UKF permanecem dentro (ou muito perto) da faixa esperada o tempo todo.

A Tabela 5 quantifica o que as Figuras 3 e 4 mostram.

Estimador	RMSE θ (°)	RMSE $\dot{\theta}$ (rad/s)	NEES médio	NEES $\leq \chi_{4,97.5\%}^2$?
Baseline	9,75	2,73	7,81	aprox. (limitrofe)
Complementar	51,62	12,89	223,17	não
KF linear	10,51	90,44	251699,13	não (diverge)
EKF	0,24	0,04	1,73	sim
UKF	0,23	0,03	1,58	sim

Tabela 5: Desempenho no *swing-up* (S2), malha fechada real ($u = -K\hat{X}$ via `swingup.m`), Monte Carlo de uma semente (`rng` fixado pelo `harness`). O RMSE de $\dot{\theta}$ do KF (90,4 rad/s) é o sintoma mais direto da divergência: o ganho calculado a partir de um modelo linearizado errado deixa de fazer sentido físico.

O KF linear **não é apenas pior**: ele diverge de forma *otimista* — sua própria covariância P relata confiança crescente exatamente quando o erro real explode (Figura 4), o que o tornaria perigoso numa malha real caso alguém confiasse em P para, por exemplo, decidir quando é seguro comutar para o LQR. O baseline (sem modelo algum) surpreendentemente não diverge tão mal quanto o KF: por não ter um modelo linearizado a "proteger", ele simplesmente segue a medição filtrada, o que é ruim (RMSE 9,75°) mas nunca catastrófico. Esse é precisamente o argumento da Introdução do relatório: o KF linear compensa *apenas* no regime quase-linear; fora dele, um modelo *errado* pode ser pior que nenhum modelo, e é exatamente aí que EKF e UKF — que carregam o modelo não-linear **correto** — se destacam.

8.5 Síntese comparativa final: cinco estimadores, seis cenários

A Tabela 6 reúne o RMSE de θ para todos os cinco estimadores em todos os seis cenários (dados gerados por `src/main.m + tests/build_report_results.m`, a partir da malha fechada real, $u = -K\hat{X}/\text{swing-up}$ via `swingup.m`).

Cenário	Baseline	Complementar	KF	EKF	UKF
S1 — Estabilização	0,58	15,58	0,24	0,25	0,24
S2 — <i>Swing-up</i>	9,75	51,62	10,51	0,24	0,23
S3 — Ruído alto	9,55	16,77	1,06	1,17	1,09
S4 — Baixa taxa/ <i>dropout</i>	0,87	17,58	0,38	0,24	0,26
S5 — Q/R mal ajustados	0,63	15,16	0,29	0,29	0,27
S6 — Inicialização ruim	1,29	15,78	0,24	0,24	0,24

Tabela 6: RMSE de θ (graus) por estimador e cenário, malha fechada real, Fase 2 completa.

Alguns pontos, além do já discutido S2:

- **EKF e UKF praticamente empatados em todos os cenários** — esperado, já que o modelo de medição é linear (Seção 8.2): a única diferença estrutural entre os dois está em como cada um propaga a **predição** não-linear, e no regime deste projeto (pêndulo de haste única, poucos graus de liberdade) essa diferença raramente é decisiva. A vantagem do UKF aparece exatamente onde a curvatura de f é maior dentro de um único passo — o *swing-up* (S2) —, onde tira uma pequena vantagem sobre o EKF (0,23° *vs.* 0,24°; mais nítida em $\dot{\theta}$, Tabela 5).
- **S1, S4, S5, S6 (regime quase-linear): os quatro estimadores "com modelo" convergem para desempenho semelhante.** Fora do *swing-up*, a vizinhança da vertical é onde a linearização do KF vale, então KF, EKF e UKF ficam a poucos centésimos de grau uns dos outros — o EKF/UKF não "perdem" nada por levar um modelo mais caro quando a situação não exige.
- **S5 (mistuned Q/R) afeta os três "com modelo" igualmente.** Como os três compartilham por padrão as mesmas covariâncias $p.Q/p.R$ (salvo *override* $p.Q_{\text{ekf}}/p.R_{\text{ekf}}$ etc., Tabela 2), o NEES médio sobe de forma quase idêntica para os três em S5 (KF 19,19, EKF 19,66, UKF 18,29, todos acima do limite χ_4^2 de 11,14) — confirma que a inconsistência estatística em S5 é uma propriedade do **ajuste de Q/R** , não do tipo de estimador: nenhuma sofisticação de modelo compensa uma covariância informada errada ao filtro.
- **Anomalia pontual: $\dot{\theta}$ do UKF em S6.** No cenário de inicialização ruim (prior 30° fora da verdade, $P_0 = 100I$), o RMSE de $\dot{\theta}$ do UKF sobe para 0,236 rad/s — bem acima do EKF (0,026) e do KF (0,045), embora o RMSE de θ permaneça idêntico aos outros (0,24°). O sintoma aparece só na velocidade angular, não observada diretamente, e é consistente com um transiente nos primeiros passos: com P_0 tão grande, os *sigma points* iniciais se espalham por uma região onde a dinâmica RK4 de um passo já não é bem resumida pela média e covariância ponderadas (a própria motivação do UKF — Seção 8.2 — é mais precisa quanto

mais suave for f dentro do espalhamento amostrado). O efeito não compromete a conclusão geral (o RMSE de θ e a convergência global não são afetados) mas é um bom candidato a investigação futura, por exemplo variando α/κ especificamente para P_0 grandes.

- **Tempo de convergência ($t_{converge}$) é uma métrica frágil perto do limiar de tolerância.** Em S3 (ruído alto), o KF converge em 3,26 s enquanto EKF e UKF aparecem como NaN (nunca convergem dentro dos 10 s simulados) — apesar de RMSE de θ **comparável ou melhor** nos três (1,06°, 1,17° e 1,09°, respectivamente). A explicação está no RMSE de $\dot{\theta}$, muito próximo da própria tolerância de convergência (0,10 rad/s): `compute_metrics.m` exige que **todos** os quatro estados fiquem simultaneamente dentro da tolerância por uma janela contínua de 1 s; quando o RMSE de um canal está colado no limiar, um único passo ruidoso acima do limite reinicia a janela, e se nenhuma janela de 1 s inteiramente limpa ocorrer nos 10 s de simulação o resultado é NaN — não porque o filtro "não convergiu" no sentido usual, mas porque o critério binário é sensível à sorte da realização de ruído perto da fronteira. O mesmo motivo explica por que baseline e complementar quase nunca convergem por este critério (Tabela 6, RMSE de $\dot{\theta}$ tipicamente acima de 0,10 rad/s mesmo em S1): a diferenciação numérica amplifica ruído de medição exatamente no canal de velocidade que a métrica exige controlar.

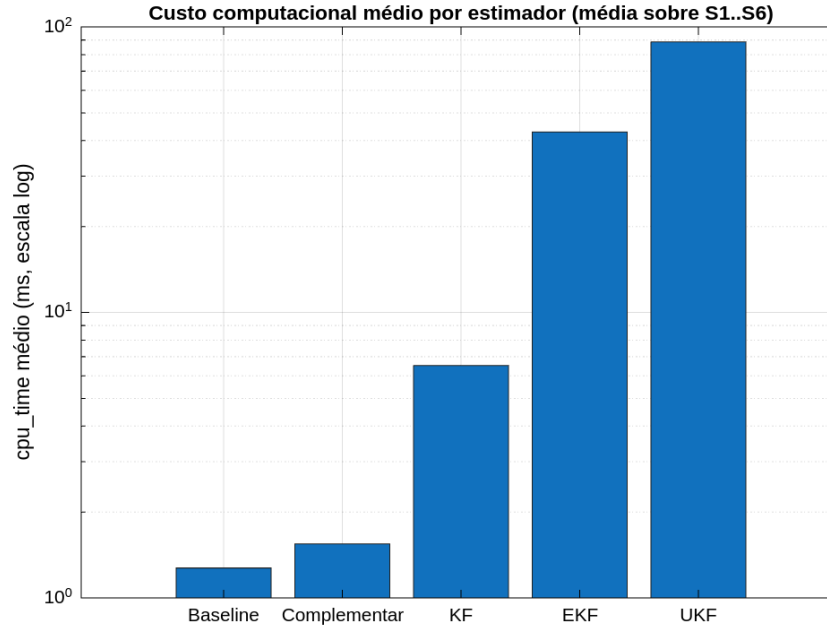


Figura 5: Custo computacional médio por estimador (`cpu_time`, escala log), medido dentro de `run_scenario.m` e médio sobre os seis cenários. A ordem reflete diretamente a complexidade algorítmica de cada método por passo: baseline e complementar são $O(1)$ (sem álgebra matricial); KF é uma predição/atualização 4×4 fixa; EKF soma o custo de `jacobian_f` + `expm` (4×4) a cada passo; UKF paga por $2n+1 = 9$ integrações RK4 completas da dinâmica não-linear por passo, além da fatoração de Cholesky.

Em termos absolutos (Figura 5), baseline e complementar custam $\sim 1,3\text{--}1,5$ ms por execução completa (todo o cenário, não por passo); o KF, $\sim 6,5$ ms ($5\times$ o baseline); o EKF, $\sim 42,8$ ms ($33\times$); e o UKF, $\sim 88,6$ ms ($68\times$ o baseline, $2,1\times$ o próprio EKF — o preço de propagar 9 pontos por RK4 em vez de 1 Jacobiano analítico). Mesmo assim, o custo absoluto do UKF continua na casa de dezenas de milissegundos para uma simulação inteira de 10–15 s a 100 Hz: **em nenhum cenário deste projeto o custo computacional é proibitivo**, mas a razão de custo ($2\times$ EKF/UKF, $7\text{--}14\times$ KF/baseline) seria o critério decisivo caso o alvo fosse um microcontrolador de malha rápida, onde o EKF ofereceria quase a mesma precisão do UKF (Tabela 6) por metade do custo por passo.

9 Arquitetura da malha e discussão

A Figura 6 reúne os módulos num diagrama de blocos da malha fechada. O laço opera em tempo discreto (passo Δt): a cada instante o **sensor** mede as posições ($z = [x; \theta]$ com ruído/perda), o **estimador** (KF, EKF ou UKF) reconstrói o estado completo $\hat{X}$, o **controlador** de duas fases gera a força u e a **planta** não-linear é integrada até o passo seguinte.

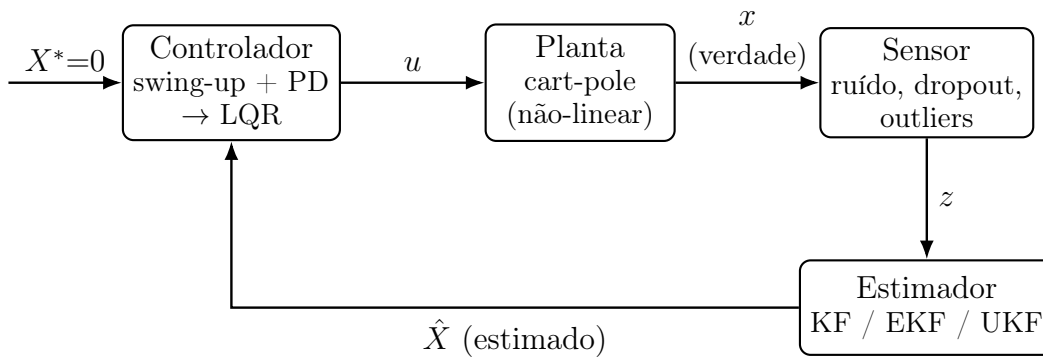


Figura 6: Malha de controle com estimação de estados. O controlador nunca acessa o estado verdadeiro: fecha o laço sobre a estimativa $\hat{X}$ (princípio da separação). É neste ponto que se comparam os cinco estimadores.

Dois aspectos merecem destaque na discussão. Primeiro, o **princípio da separação**: o controlador foi projetado supondo acesso ao estado, mas na prática opera sobre $\hat{X}$; a qualidade de cada estimador reflete-se diretamente no desempenho da malha, e é exatamente essa degradação que o trabalho quantifica. A Seção 8.5 mostra essa degradação de forma concreta: com o estimador baseline ou o complementar, o próprio LQR — projetado e verificado como estável em 4.4 — herda o erro de estimação e nunca alcança o desempenho da malha ideal; com KF/EKF/UKF no regime em que cada um é válido, a malha real se aproxima da malha com estado verdadeiro. Segundo, o **controlador de duas fases**: longe da vertical usa a lei não-linear de *energy shaping* com o PD de carro; dentro da janela de captura comuta para o LQR linear. Assim, um único experimento leva os filtros por um regime fortemente não-linear (*swing-up*) e por um regime quase-linear (estabilização), evidenciando onde cada técnica de estimação compensa — o resul-

tado mais nítido do trabalho sendo, precisamente, a divergência do KF linear e a robustez de EKF/UKF durante o *swing-up* (Seção 8.4).

Conclusão

Este projeto implementou e comparou, dentro de uma malha de controle fechada real, quatro níveis de estimadores de estado — diferenciação com passa-baixas, filtro complementar, KF linear, EKF e UKF — aplicados ao pêndulo invertido sobre carro. As conclusões centrais, sustentadas pelos seis cenários (S1–S6) rodados com o harness completo (`src/main.m`):

- **O KF linear é ótimo apenas dentro da vizinhança onde foi linearizado.** No regime quase-linear (estabilização, S1/S4/S5/S6) ele empata com EKF/UKF; fora dele (*swing-up*, S2) ele diverge de forma **otimista** — a covariância relatada encolhe exatamente quando o erro real explode (NEES $\sim 2,5 \times 10^5$ contra um limite χ_4^2 de 11,14) — o que o torna não apenas impreciso, mas estatisticamente enganoso.
- **EKF e UKF resolvem exatamente esse problema**, carregando o modelo não-linear *correto* (via Jacobiano *online* ou *sigma points*) em vez de uma aproximação fixa: ambos mantêm RMSE de θ abaixo de $0,25^\circ$ e NEES dentro da faixa esperada em **todos** os seis cenários, inclusive o *swing-up*. A diferença entre os dois é pequena neste projeto (medição linear, poucos graus de liberdade) — o UKF leva uma vantagem marginal em regimes fortemente não-lineares, ao custo de $\sim 2\times$ o tempo de CPU do EKF.
- **"Mais modelo" nem sempre significa "melhor"**: o filtro complementar de ganho fixo é o pior estimador em praticamente todo cenário testado, e mesmo o baseline sem modelo algum supera o KF linear divergente no *swing-up* — um lembrete de que um modelo *errado* pode ser pior do que nenhum modelo.
- **O princípio da separação se verifica na prática, não só na teoria**: o mesmo ganho LQR, estável por projeto (Seção 4.4), produz malhas fechadas de qualidade muito diferente dependendo de qual estimador alimenta $\hat{X}$ — a malha real degrada junto com o estimador, e não apenas o RMSE do estimador isolado.
- Testes de consistência estatística (NEES/NIS) revelaram uma falha que o RMSE sozinho esconde: em Q/R mal ajustados (S5), o RMSE quase não piora, mas os três filtros baseados em modelo passam a relatar uma incerteza menor que a real — só o teste χ^2 expõe esse problema.

Como trabalho futuro, ficam abertos: a recalibração de Q pelo método de Van Loan [17] (em vez do ajuste direto em tempo discreto, Seção 7.3.2); a adoção uniforme da forma de Joseph na atualização do UKF (Seção 8.2); e a investigação da anomalia pontual de $\dot{\theta}$ do UKF sob inicialização ruim (S6, Seção 8.5).

Referências

- [1] MathWorks. *Derive and Simulate Cart-Pole System* (Symbolic Math Toolbox). Disponível em: <https://www.mathworks.com/help/symbolic/derive-and-simulate-cart-pole-system.html>.
- [2] R. Tedrake. *Underactuated Robotics: Algorithms for Walking, Running, Swimming, Flying, and Manipulation* — Cap. *The Simple Pendulum*. Disponível em: <https://underactuated.mit.edu/pend.html>.
- [3] K. J. Åström, K. Furuta. *Swinging up a Pendulum by Energy Control*. Automatica, vol. 36, n. 2, pp. 287–295, 2000.
- [4] R. Tedrake. *Underactuated Robotics* — Cap. *Acrobats, Cart-Poles, and Quadrotors*. Disponível em: <https://underactuated.mit.edu/acrobot.html>.
- [5] C. C. Chung, J. Hauser. *Nonlinear control of a swinging pendulum*. Automatica, vol. 31, n. 6, pp. 851–862, 1995. Ver também M. W. Spong, *Energy Based Control of a Class of Underactuated Mechanical Systems*, IFAC World Congress, 1996.
- [6] R. E. Kalman. *A New Approach to Linear Filtering and Prediction Problems*. Journal of Basic Engineering (ASME Transactions), vol. 82, Series D, pp. 35–45, 1960.
- [7] G. Welch, G. Bishop. *An Introduction to the Kalman Filter*. Technical Report TR 95-041, University of North Carolina at Chapel Hill, 1995 (atualizado em 2006). Disponível em: https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf.
- [8] D. Simon. *Optimal State Estimation: Kalman, H_∞ , and Nonlinear Approaches*. Wiley, 2006.
- [9] M. S. Grewal, A. P. Andrews. *Kalman Filtering: Theory and Practice Using MATLAB*, 4.^a ed. Wiley, 2015 (1.^a ed. 1993).
- [10] S. J. Julier, J. K. Uhlmann. *A New Extension of the Kalman Filter to Nonlinear Systems*. Proc. SPIE, Signal Processing, Sensor Fusion, and Target Recognition VI, vol. 3068, pp. 182–193, 1997.
- [11] E. A. Wan, R. van der Merwe. *The Unscented Kalman Filter for Nonlinear Estimation*. Proc. IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium (AS-SPCC), pp. 153–158, 2000.
- [12] W. T. Higgins. *A Comparison of Complementary and Kalman Filtering*. IEEE Transactions on Aerospace and Electronic Systems, vol. AES-11, n. 3, pp. 321–325, 1975.
- [13] R. G. Brown, P. Y. C. Hwang. *Introduction to Random Signals and Applied Kalman Filtering*, 4.^a ed. Wiley, 2012.
- [14] G. F. Franklin, J. D. Powell, M. L. Workman. *Digital Control of Dynamic Systems*, 3.^a ed. Addison-Wesley, 1997.

- [15] K. J. Åström, B. Wittenmark. *Computer-Controlled Systems: Theory and Design*, 3.^a ed. Prentice Hall, 1997.
- [16] A. V. Oppenheim, R. W. Schaffer. *Discrete-Time Signal Processing*, 3.^a ed. Prentice Hall, 2009.
- [17] C. F. Van Loan. *Computing Integrals Involving the Matrix Exponential*. IEEE Transactions on Automatic Control, vol. 23, n. 3, pp. 395–404, 1978.
- [18] Y. Bar-Shalom, X. R. Li, T. Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.